Explaining Defeasible Entailments: A Pedagogical User Interface for Rational Closure

Nikita Martin
mrtnik003@myuct.ac.za
University of Cape Town
Cape Town, South Africa

Abstract

Defeasible reasoning is a form of non-monotonic reasoning that allows conclusions to be revised when new or more specific information is introduced, closely mirroring how humans reason in everyday life. Despite its growing importance in AI and knowledge representation, defeasible reasoning has not yet been widely incorporated into logic education, and existing defeasible reasoning tools provide results and explanations but offer limited support for step-by-step inspection of the intermediate computational states of the reasoning process. This paper presents a web-based explanation tool for defeasible reasoning, developed as part of a broader collaborative system that also includes Lexicographic and Relevant Closure, implemented by other group members. The tool provides a debugger-style step-through interface that exposes the BaseRank construction and Rational Closure entailment process, presenting computational states alongside plain-English explanations and formal pseudocode. A dedicated drowning problem example explains a known limitation of Rational Closure, while a comparison mode allows users to observe how Rational Closure, Lexicographic Closure, and Relevant Closure process the same knowledge base and query. A guided tutorial further supports exploration of the system. The tool is evaluated through manual verification, automated unit testing, comparison with existing implementations, comparative analysis of the supported entailment methods, execution-time observations, and informal expert feedback. The results provide evidence that the implementation behaves consistently with the theoretical algorithms and that the interface makes the intermediate reasoning process easier to inspect and interpret.

Keywords

Propositional Logic, Defeasible Reasoning, Rational Closure, Pedagogical Tools, User Interface

1 Introduction

Artificial Intelligence systems rely on Knowledge Representation and Reasoning (KRR), a subfield concerned with encoding domain knowledge and deriving new conclusions from it to support decision-making. Traditionally, KRR has used classical logic as its core reasoning mechanism [2, 18]. While classical logic provides a solid foundation for reasoning, its monotonic nature means that conclusions cannot be retracted when new information is introduced [14, 15]. This makes classical logic poorly suited to reasoning with exceptions, where more specific information may require previously drawn conclusions to be revised. For example, if we represent the generalisation that "all birds fly", together with the facts that penguins are birds and penguins do not fly, classical reasoning produces an inconsistency rather than treating penguins as an exception [14].

To address this limitation, defeasible reasoning adopts a non-monotonic approach in which conclusions are drawn tentatively and may be revised when new or more specific information is provided [14, 21], more closely mirroring how humans reason in everyday life [14]. The KLM framework introduced by Kraus, Lehmann, and Magidor [15] provides a formal foundation for defeasible reasoning by defining a set of properties that characterise rational patterns of non-monotonic reasoning.

The conclusions derived from a knowledge base through logical reasoning are known as entailments [23]. In classical logic, entailment captures whether a conclusion necessarily follows from the available knowledge, while defeasible entailment extends this notion to non-monotonic reasoning, where conclusions may depend on typicality and exceptions. Key defeasible entailment algorithms include Rational Closure [5], Lexicographic Closure [4], and Relevant Closure [3]. Rational Closure is the primary entailment method considered in this paper, while Lexicographic and Relevant Closure are included for comparison.

Despite its growing importance in AI, there remains limited pedagogical support for teaching and exploring defeasible reasoning. Defeasible reasoning algorithms may be difficult to interpret for users who are familiar with classical logic but are encountering non-monotonic reasoning for the first time. Wang notes that conclusions produced by defeasible reasoning tools can be difficult to understand due to the volume and complexity of the statements involved [23]. Existing tools can compute defeasible entailment, but computing a result is not the same as helping a learner understand how that result was reached. In particular, there remains limited support for incrementally exploring the computational steps involved in defeasible reasoning. This motivates investigating whether a structured step-through presentation can make Rational Closure easier to inspect and interpret.

This leads to the following research question: **RQ:** How can Rational Closure be presented in a way that makes its intermediate reasoning process easier to inspect and interpret, while also supporting comparison with other defeasible entailment methods?

To investigate this question, this paper presents a web-based pedagogical step-through tool for defeasible reasoning, focusing on the Rational Closure algorithm. The tool forms part of a broader collaborative system that also includes Lexicographic Closure [4] and Relevant Closure [3], implemented by other group members. Built on the TweetyProject framework [22] using a Spring Boot backend and a React frontend written in TypeScript, the tool exposes the BaseRank construction and Rational Closure entailment process through a debugger-style step-through interface. Each computational step is presented alongside plain-English explanations

and formal pseudocode, helping users connect the formal algorithm with its execution on a knowledge base.

This paper focuses on the Rational Closure component of the broader PDR system, together with the shared BaseRank functionality on which it operates. Its main contribution is a Rational Closure debugger-style step-through interface that exposes the intermediate computational states through formal pseudocode, plain-English explanations, and justification-based explanations for exceptionality checks. This environment is further supported by a dedicated drowning problem example illustrating a known limitation of Rational Closure, as well as a comparison mode that allows users to observe how Rational Closure, Lexicographic Closure, and Relevant Closure differ on the same knowledge base and query. The broader system also includes a shared guided tutorial that walks users through the drowning problem end to end. The tool is evaluated in terms of correctness, pedagogical support, comparative behaviour, execution-time observations, and informal expert feedback. Together, these features and evaluation criteria address the research question by supporting the inspection and interpretation of the intermediate Rational Closure reasoning process, while enabling comparison with other defeasible entailment methods.

The remainder of the paper is structured as follows: Section 2 provides background, Section 3 discusses related work, Section 4 describes the system design and implementation, Section 5 presents the results, analysis, and discussion, and Section 6 concludes.

2 Background

2.1 Propositional Logic

Propositional Logic (PL) is a formal system for representing and reasoning about knowledge. Simple statements are represented as propositional atoms. For example, the statement "is a bird" can be represented by the propositional atom *bird*, while "flies" can be represented as *flies*. These atoms can be combined using logical connectives, such as negation ($\neg$), conjunction ($\wedge$), disjunction ($\vee$), and implication ($\rightarrow$), to form formulas [1, 9]. The semantics of PL are defined through valuation functions $u : P \rightarrow \{T, F\}$ that assign truth values to propositional atoms. A valuation u satisfies a formula α, written $u \Vdash \alpha$, when α evaluates to true under u. A formula is satisfiable if there exists at least one valuation under which it is true, and unsatisfiable if no such valuation exists [14].

A knowledge base $\mathcal{K}$ is a finite set of formulas that represent information about a domain, for example,

$$\mathcal{K} = \{bird \rightarrow flies,\ bird \rightarrow wings,\ penguin \rightarrow bird\}$$

[9, 14]. The notion of satisfiability extends to a knowledge base $\mathcal{K}$, which is satisfiable if there exists a valuation that satisfies every formula in $\mathcal{K}$, denoted by $u \Vdash \mathcal{K}$ [14]. Once a knowledge base has been defined, the next question is which conclusions follow from it. This is captured by the notion of entailment.

In propositional logic, this is referred to as classical entailment. Classical entailment describes when a conclusion necessarily follows from the available knowledge. For a knowledge base $\mathcal{K}$ and a formula α, α is entailed by $\mathcal{K}$, written as $\mathcal{K} \models \alpha$, if every valuation that satisfies $\mathcal{K}$ also satisfies α [14]. For example, consider the knowledge base

$$\mathcal{K} = \{penguin \rightarrow bird,\ bird \rightarrow animal\}.$$

The query $penguin \rightarrow animal$ is entailed by $\mathcal{K}$ because any valuation satisfying both formulas in the knowledge base must also satisfy $penguin \rightarrow animal$.

Classical entailment can equivalently be checked using satisfiability. To determine whether $\mathcal{K} \models \alpha$, the negation of the query is added to the knowledge base and the resulting set $\mathcal{K} \cup \{\neg\alpha\}$ is tested for satisfiability. If the set is unsatisfiable, then there is no interpretation in which all formulas in $\mathcal{K}$ are satisfied while α is false, and therefore $\mathcal{K} \models \alpha$ [9, 14, 23]. This reduction of entailment to satisfiability checking is particularly important in this work because classical entailment checks are used to determine exceptionality in the BaseRank and Rational Closure algorithms.

A fundamental property of classical reasoning is monotonicity, whereby any conclusion entailed by a knowledge base cannot be retracted when new information is added [9, 14]. This becomes limiting in domains where exceptions may arise and require conclusions to be revised [14]. Non-monotonic reasoning addresses this limitation by allowing conclusions to be withdrawn when additional information is introduced [14]. Defeasible reasoning is a form of non-monotonic reasoning and is discussed in the following section.

2.2 Defeasible Reasoning

Defeasible reasoning is a form of non-monotonic reasoning in which conclusions are held tentatively and may be revised when more specific information becomes available [21]. Central to defeasible reasoning is the notion of typicality, which captures what is typically or normally the case rather than expressing only absolute rules [14, 15]. Classical logic uses universal statements such as $bird \rightarrow flies$, which cannot naturally represent exceptions such as penguins. Defeasible reasoning instead allows for generalisations such as "birds typically fly", which hold in the general case but may be overridden by more specific information [14].

The KLM framework, introduced by Kraus, Lehmann, and Magidor [15], provides a formal foundation for defeasible reasoning by extending propositional logic with the defeasible conditional $\alpha \mathrel{|\!\sim} \beta$, read as "$\alpha$ typically implies β". This differs from the classical implication $\alpha \rightarrow \beta$, which means that whenever α is true, β must also be true [14, 15]. The framework defines a set of rationality properties, known as the KLM properties, that constrain how defeasible entailment should behave in the presence of exceptions [15]. For example, the defeasible statement $bird \mathrel{|\!\sim} flies$ expresses that "birds typically fly", allowing exceptions such as penguins, whereas $bird \rightarrow flies$ expresses that all birds fly and does not allow the same exception.

A defeasible knowledge base contains both defeasible and classical statements, for example:

$$\mathcal{K} = \{bird \mathrel{|\!\sim} flies,\ bird \mathrel{|\!\sim} wings,\ penguin \rightarrow bird,\ penguin \mathrel{|\!\sim} \neg flies\}$$

where $bird \mathrel{|\!\sim} flies$ and $bird \mathrel{|\!\sim} wings$ are defeasible statements expressing typical properties of birds, $penguin \rightarrow bird$ is a classical statement expressing that all penguins are birds, and $penguin \mathrel{|\!\sim} \neg flies$ expresses that penguins typically do not fly. Defeasible entailment, written $\mathcal{K} \mathrel{|\!\approx} \alpha \mathrel{|\!\sim} \beta$, holds when β typically follows from α according to $\mathcal{K}$ [14, 15]. For example, $\mathcal{K} \mathrel{|\!\approx} penguin \mathrel{|\!\sim} \neg flies$ expresses that, according to the knowledge base, penguins typically do not fly.

Defeasible entailment relations satisfying the KLM properties can be characterised using ranked interpretations, where valuations are ranked according to their level of typicality [5, 14]. This paper focuses on Rational Closure [5], an algorithm for computing defeasible entailment that satisfies the KLM properties.

A key concept used in determining defeasible entailment is *exceptionality*. To determine exceptionality, defeasible statements are first materialised by converting each defeasible statement $\alpha \mathrel{|\!\sim} \beta$ into its corresponding classical implication $\alpha \rightarrow \beta$, formally defined as:

$$\vec{\mathcal{K}} \equiv_{def} \{\alpha \rightarrow \beta \mid \alpha \mathrel{|\!\sim} \beta \in \mathcal{K}\}$$

An antecedent α is exceptional with respect to the knowledge base $\mathcal{K}$ if the materialised knowledge base classically entails $\neg\alpha$, that is, $\vec{\mathcal{K}} \models \neg\alpha$ [5, 14]. This means there is no valuation satisfying the materialised knowledge base in which α is true. In the running example, *penguin* is exceptional because the materialised knowledge base entails $\neg penguin$. This notion of exceptionality forms the basis of the BaseRank construction discussed in the following section [5, 14].

2.3 Base Rank

BaseRank provides the ranked structure used by Rational Closure and other supported defeasible entailment methods, forming the common starting point for reasoning over the same knowledge base. To compute the defeasible entailment under Rational Closure, the knowledge base $\mathcal{K}$ must first be partitioned into a sequence of ranked sets based on the exceptionality of its antecedents [5, 8]. This partitioning is carried out by the BaseRank algorithm [5], which builds on the ranking construction proposed by Freund [8].

The algorithm begins by materialising $\mathcal{K}$ to obtain $\vec{\mathcal{K}}$, and iteratively computes a sequence of sets $\mathcal{E}_0^{\mathcal{K}}, \mathcal{E}_1^{\mathcal{K}}, \ldots$, where $\mathcal{E}_0^{\mathcal{K}} = \vec{\mathcal{K}}$ and each $\mathcal{E}_{i+1}^{\mathcal{K}}$ contains only those statements whose antecedents are exceptional with respect to $\mathcal{E}_i^{\mathcal{K}}$ [5]. Here, $\mathcal{E}_i^{\mathcal{K}}$ represents the statements that remain under consideration at iteration i, while $\mathcal{R}_i$ contains the statements assigned to the rank at that iteration. At each iteration, non-exceptional statements are assigned to the current rank $\mathcal{R}_i = \mathcal{E}_i^{\mathcal{K}} \setminus \mathcal{E}_{i+1}^{\mathcal{K}}$, while exceptional ones are carried to the next iteration. This process terminates when $\mathcal{E}_{i-1}^{\mathcal{K}} = \mathcal{E}_i^{\mathcal{K}}$, at which point the remaining statements are assigned to $\mathcal{R}_\infty$ [5].

Example. Consider the knowledge base:

$$\mathcal{K} = \{bird \mathrel{|\!\sim} flies, bird \mathrel{|\!\sim} wings, penguin \rightarrow bird, penguin \mathrel{|\!\sim} \neg flies\}$$

Applying the BaseRank algorithm produces the following:

- **Rank 0 ($\mathcal{R}_0$):** The statements $bird \mathrel{|\!\sim} flies$ and $bird \mathrel{|\!\sim} wings$ are assigned to the lowest rank because the antecedent *bird* is not exceptional at the initial level.
- **Rank 1 ($\mathcal{R}_1$):** The antecedent *penguin* is exceptional at the initial level, but becomes non-exceptional after the Rank 0 statements have been assigned and removed from the set under consideration. The statement $penguin \mathrel{|\!\sim} \neg flies$ is therefore assigned to Rank 1.
- **Rank ∞ ($\mathcal{R}_\infty$):** The classical statement $penguin \rightarrow bird$ remains when the BaseRank process terminates and is therefore assigned to $\mathcal{R}_\infty$.

The resulting table is summarised in Table 1.

Rank	Statements
$\mathcal{R}_\infty$	$penguin \rightarrow bird$
$\mathcal{R}_1$	$penguin \mathrel{|\!\sim} \neg flies$
$\mathcal{R}_0$	$bird \mathrel{|\!\sim} flies,\ bird \mathrel{|\!\sim} wings$

Table 1: BaseRank partition of $\mathcal{K}$

The ranking produced by BaseRank reflects the level of exceptionality of statement antecedents. Lower ranks contain statements whose antecedents become non-exceptional earlier, while higher ranks contain statements whose antecedents remain exceptional for longer. This ranking now provides the structure needed by Rational Closure to determine which information should be discarded when answering a defeasible query.

Algorithm 1 BaseRank [5]

Input: A defeasible knowledge base $\mathcal{K}$
Output: An ordered tuple $(R_0, \ldots, R_{n-1}, R_\infty, n)$
$\vec{\mathcal{K}} = Materialisation(\mathcal{K})$
$E_0 := \vec{\mathcal{K}}$
$i := 0$
repeat
  $E_{i+1} := \{\alpha \rightarrow \beta \in E_i \mid E_i \models \neg\alpha\}$
  $R_i := E_i \setminus E_{i+1}$
  $i := i + 1$
until $E_{i-1} = E_i$
$R_\infty := E_{i-1}$
if $E_{i-1} = \emptyset$ **then**
  $n := i - 1$
else
  $n := i$
end if
return $(R_0, \ldots, R_{n-1}, R_\infty, n)$

2.4 Rational Closure

Rational Closure, first defined by Lehmann and Magidor [17], is a method for computing defeasible entailment from a knowledge base. It provides an important point of departure for later, bolder entailment approaches such as Lexicographic and Relevant Closure [3, 4]. Given the ranking produced by BaseRank, Rational Closure determines whether a defeasible query $\alpha \mathrel{|\!\sim} \beta$ is entailed by $\mathcal{K}$ by progressively removing the lowest ranks until the antecedent of the query is no longer exceptional, before performing a final classical entailment check [5, 6].

The finite ranks are first combined into a working set $\mathcal{R}$, while $\mathcal{R}_\infty$ is retained throughout the reasoning. While $\mathcal{R}_\infty \cup \mathcal{R} \models \neg\alpha$, meaning that the query antecedent remains exceptional, the lowest rank is removed from $\mathcal{R}$. Removing lower-ranked information allows the algorithm to reason within progressively more exceptional contexts. This process continues until α is no longer exceptional or no finite ranks remain, after which the final entailment check determines whether $\mathcal{R}_\infty \cup \mathcal{R} \models \alpha \rightarrow \beta$ [5].

Example. Continuing with $\mathcal{K}$ from the previous example, consider the query $penguin \mathrel{|\!\sim} \neg flies$. The BaseRank partition gives:

$\mathcal{R}_0 = \{bird \mid\sim flies,\ bird \mid\sim wings\}$, $\mathcal{R}_1 = \{penguin \mid\sim \neg flies\}$, and $\mathcal{R}_\infty = \{penguin \rightarrow bird\}$. Since *penguin* is exceptional with respect to $\mathcal{R}_\infty \cup \mathcal{R}$, $\mathcal{R}_0$ is removed. After removal, *penguin* is no longer exceptional, and the final check confirms that $\mathcal{R}_\infty \cup \mathcal{R}_1 \models penguin \rightarrow \neg flies$. Therefore, $\mathcal{K} \mid\approx_{RC} penguin \mid\sim \neg flies$ holds.

Algorithm 2 RationalClosure [5]

Input: A knowledge base $\mathcal{K}$ and a query $\alpha \mid\sim \beta$
Output: **true** if $\mathcal{K} \mid\approx_{RC} \alpha \mid\sim \beta$, and **false** otherwise
$(\mathcal{R}_0, \ldots, \mathcal{R}_{n-1}, \mathcal{R}_\infty, n) :=$ BaseRank($\mathcal{K}$)
$i := 0$
$\mathcal{R} := \bigcup_{j=0}^{n-1} \mathcal{R}_j$
while $\mathcal{R}_\infty \cup \mathcal{R} \models \neg\alpha$ **and** $\mathcal{R} \neq \emptyset$ **do**
    $\mathcal{R} := \mathcal{R} \setminus \mathcal{R}_i$
    $i := i + 1$
end while
return $\mathcal{R}_\infty \cup \mathcal{R} \models \alpha \rightarrow \beta$

The limitation of Rational Closure is that entire ranks are removed at once when resolving exceptionality. As a result, statements that are unrelated to the source of exceptionality may be discarded together with the conflicting information, which can affect the final entailment result. This limitation is known as the *drowning problem* [4, 5]. To illustrate, consider the query $penguin \mid\sim wings$. Although the knowledge base contains the classical statement $penguin \rightarrow bird$ and the defeasible statement $bird \mid\sim wings$, the statement $bird \mid\sim wings$, which is unrelated to the source of exceptionality, is removed alongside $bird \mid\sim flies$ during rank removal, as both belong to Rank 0. The final check fails, and $\mathcal{K} \mid\approx_{RC} penguin \mid\sim wings$ does not hold. This limitation motivates entailment algorithms such as Lexicographic and Relevant Closure, which preserve or consider ranked information more selectively [3, 4].

2.5 Other Defeasible Algorithms

Rational Closure is one of the central KLM-style methods for computing defeasible entailments, but its whole-rank removal strategy can lead to the drowning problem. Lexicographic Closure, proposed by Lehmann [16], addresses the drowning problem by preserving more information than Rational Closure when conflicts occur. Rather than discarding an entire conflicting rank, Lexicographic Closure weakens the rank so that more information can be preserved for the entailment check. For example, while Rational Closure does not conclude that penguins have wings because $bird \mid\sim wings$ is removed alongside $bird \mid\sim flies$, Lexicographic Closure can derive this conclusion because the unrelated wing information is preserved. Lexicographic Closure strictly extends Rational Closure, whereby every inference drawn by Rational Closure is also drawn by Lexicographic Closure, but not vice versa [4, 16].

Relevant Closure, proposed by Casini et al. [3], takes a different approach by focusing on statements relevant to a query and its exceptionality. Rather than removing all information from an exceptional rank, Relevant Closure restricts rank removal to information relevant to the conflict, allowing unrelated statements to remain available for the entailment checks. This can recover conclusions that may be lost under Rational Closure through whole-rank removal [3].

Both algorithms are implemented by other group members as part of a broader PDR system. These differences are central to the comparison environment, which allows users to observe how Rational Closure, Lexicographic Closure, and Relevant Closure treat the same knowledge base and query, and how those differences affect the final entailment result.

2.6 Justification of Entailments

Justifications explain entailments by identifying a minimal subset of a knowledge base that still entails the query, such that removing any formula from the subset causes the entailment to fail [11, 12, 23]. This allows an entailment to be explained using a minimal set of formulas sufficient to support it, rather than the complete knowledge base.

In defeasible reasoning, this must also account for the behaviour of the entailment algorithm, since Rational Closure may discard lower-ranked statements before performing its final classical entailment check [5]. Wang therefore adapts classical justifications to defeasible entailment by computing justifications only from formulas that remain applicable after this rank removal, and converting the resulting formulas back to defeasible form [23]. Hamayobe similarly distinguishes weak justifications based on their final classical entailment from stronger explanations that capture more of the reasoning process [11].

3 Related Work

Existing pedagogical tools for logic focus primarily on classical reasoning. Iltis [10] provides a web-based task environment for propositional logic with structured feedback, while LogicLearner [13] provides a step-by-step interactive proof environment backed by an AI proof solver. These tools demonstrate the use of interactive and feedback-driven interfaces for teaching logic, but neither addresses defeasible reasoning.

Beyond pedagogical tools for classical logic, several tools have been developed to implement defeasible reasoning algorithms. The TweetyProject [22] is a comprehensive Java library for knowledge base representation and reasoning, supporting over fifteen formalisms, including defeasible reasoning. While it is widely used as a foundation for building reasoning tools, it does not provide a predefined pedagogical interface; interaction is primarily through its Java APIs, with no step-through guidance or explanatory interface aimed at learners.

EXTRC, developed by Moloi [19] and subsequently extended by Cotterrell [7], implements Rational Closure and Lexicographic Closure using the TweetyProject library, presenting defeasible entailment results alongside base rank partitions and execution times. While the tools provide both mathematical and plain-English explanations of the resulting entailment, they do not expose the intermediate reasoning process incrementally. Users cannot follow the construction of the BaseRank or the removal of ranks during entailment checking one step at a time. Hamayobe [11] implements Rational Closure with justification-based explanations identifying the statements that support a conclusion. While this provides more information about why a conclusion holds, the reasoning is presented as a complete explanation on a single page rather than as a walkthrough of the algorithm's execution.

Pedagogical tools such as Iltis and LogicLearner demonstrate the value of structured interactive environments for teaching logic, but do not address defeasible reasoning. Existing defeasible reasoning tools such as EXTRC and Hamayobe provide reasoning results and explanations, but do not expose the execution of the reasoning algorithms incrementally through a step-through interface. This gap motivates the development of PDR.

4 System Design and Implementation

This section describes the design and implementation of the pedagogical step-through tool for defeasible reasoning, focusing on the BaseRank and Rational Closure interfaces and the comparison environment. The main individual contribution of this paper is the Rational Closure pedagogical environment, comprising the Rational Closure step-through interface, justification-based explanations for exceptionality checks, the dedicated drowning problem example and explanation, and the comparison environment for examining Rational Closure, Lexicographic Closure, and Relevant Closure. Shared group contributions of the broader collaborative system, including the BaseRank functionality, execution-time support, a guided tutorial, and an information page, support these features.

The implementation builds on existing work from EXTRC [7, 19] and Hamayobe [11], with components adapted and extended where necessary. The justification functionality used for exceptionality checks and final entailment explanations was adapted from the classical justification approach described by Wang [23]. All modifications and original contributions are clearly identified throughout this section. The system exposes BaseRank construction, exceptionality checks, rank removal during Rational Closure, and points of divergence between entailment methods, supporting the pedagogical goal of making the reasoning process visible.

4.1 System Architecture

The system follows a layered client-server architecture, separating the reasoning logic from the user interface. A high-level overview of the system is shown in Figure 1, with a larger version provided in Appendix A.1.

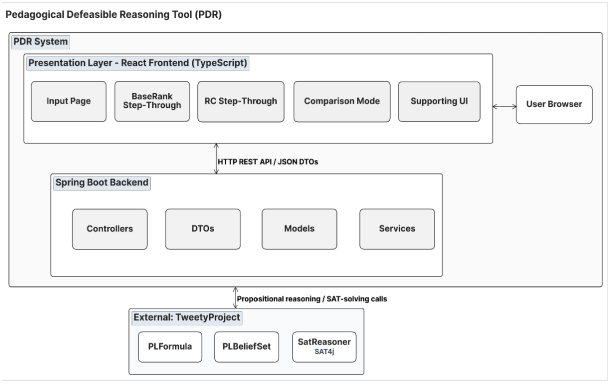


Figure 1: System Architecture of the Pedagogical Defeasible Reasoning Tool

The backend is implemented in Java using Spring Boot and contains the reasoning functionality for BaseRank and the supported defeasible entailment methods. Logical inference is performed using the TweetyProject library [22], which provides the propositional reasoning and SAT-solving functionality required for exceptionality and entailment checking. The supported reasoning methods operate through a common interface, allowing Rational Closure, Lexicographic Closure, and Relevant Closure to be used consistently within the same system.

A key architectural addition is the trace capture of intermediate execution states. Rather than returning only the final entailment result, the backend records important stages of the reasoning process, including exceptionality checks, working-set changes, and rank removal. These traces form the basis of the step-through interfaces and comparison environment. The frontend presents these recorded states through the step-through visualisations, pseudocode highlighting, and plain-English explanations.

Communication between the frontend and backend is handled through a REST API, with further architectural and implementation details provided in Appendix A.

4.2 Knowledge Base and Queries

The system accepts a propositional defeasible knowledge base $\mathcal{K}$ containing both defeasible and classical statements, together with an entailment query expressed in the same notation. Internally, the defeasible statements are materialised into classical implications where required for satisfiability and exceptionality checking [5]. The resulting representation provides the input for BaseRank and supported defeasible entailment algorithms.

4.3 Base Rank Implementation

The BaseRank algorithm [5] is a shared component of the PDR system, adapted from EXTRC [7, 19] and aligned with the theoretical algorithm of Casini and Meyer [5]. TweetyProject's `SatReasoner` and `Sat4jSolver` perform the classical entailment checks required during exceptionality testing [22]. The algorithm takes a knowledge base $\mathcal{K}$ as input and produces an ordered ranking $(R_0, \ldots R_{n-1}, R_\infty)$.

A key addition to the implementation was a trace capture mechanism that records the intermediate steps of the algorithm at each iteration, supporting the pedagogical interface. The trace captures the antecedents being tested for exceptionality, the outcome of each check, the statements assigned to the current rank, and the statements carried forward to the next iteration. Exceptionality checks are grouped by antecedent, as multiple defeasible statements may share the same antecedent, reflecting the structure of the algorithm. This information is returned to the frontend and used to reconstruct the BaseRank construction step-by-step.

The trace data drives the BaseRank step-through interface, exposing the algorithm's execution to the user one step at a time. The step-through begins by explicitly showing the materialisation of the knowledge base, where each defeasible statement is displayed alongside its classical counterpart $\alpha \rightarrow \beta$. As the user steps through the algorithm, the exceptionality check for each antecedent is shown together with the resulting rank assignments. The ranking visualiser is populated progressively as statements are assigned, while the corresponding pseudocode line is highlighted and accompanied

by plain-English explanations of the current operation. This allows users to observe the ranking as it is constructed rather than being shown only the final completed result. An example of the BaseRank step-through interface is shown in Appendix D, Figure 9.

4.4 Rational Closure Implementation

The Rational Closure algorithm [5], presented in Algorithm 2, is implemented in the `RationalReasonerImpl` class. Although the initial structure was adapted from EXTRC [7], it was largely redeveloped to follow the theoretical algorithm by Casini and Meyer [5] more closely. Given the BaseRank ranking and a query $\alpha \mid\sim \beta$, the algorithm iteratively tests the exceptionality of the query antecedent, removes lower ranks where required, and performs the final entailment check.

To support the step-through interface, the implementation records the state of the Rational Closure process at each iteration, including the current working set, the exceptionality result for the query antecedent, and any rank removed at that iteration. These recorded states are returned to the frontend, where they are used to construct the Rational Closure step-through interface.

The ranking visualiser displays all the finite ranks at the top, with $\mathcal{R}_\infty$ retained throughout. As the user steps through the algorithm, the exceptionality of the query antecedent is checked at each iteration. When it is found to be exceptional, the lowest rank is highlighted and greyed out with strikethrough, making the rank removal visible in real time. This continues until the antecedent is no longer exceptional, at which point the final classical entailment check is performed, and the result is displayed according to whether the query is entailed. At the final entailment check, the interface also presents a weak justification indicating the formulas that support the final classical entailment result. At each stage, the corresponding line of pseudocode is highlighted and accompanied by a plain-English explanation of the current operation.

A `Show Details` option is also available during exceptionality checks. When the query antecedent is found to be exceptional, the user can expand the panel to view a classical justification, adapted from Wang [23], identifying a minimal subset of the current materialised working set sufficient to support the exceptionality result. This allows users to inspect which formulas logically contribute to the check rather than relying only on syntactic relevance.

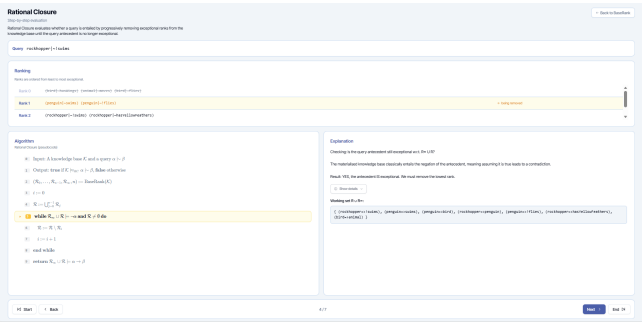


Figure 2: Rational Closure step-through interface showing an exceptionality check and its justification details.

Figure 2 illustrates the Rational Closure step-through interface during an exceptionality check, with the current algorithm state, pseudocode highlighting, and justification details visible to the user.

When the predefined drowning problem example is loaded and Rational Closure does not entail the query, the interface displays a dedicated explanation showing how whole-rank removal causes the result. This provides a concrete demonstration of the limitation and motivates the comparison with Lexicographic and Relevant Closure. Together, these features make the intermediate stages of Rational Closure directly inspectable. A larger view of the complete Rational Closure step-through interface is provided in Appendix D, Figure 14.

4.5 Comparison Mode

The comparison mode allows users to observe how Rational Closure, Lexicographic Closure, and Relevant Closure process the same knowledge base and query. It is accessed through the `Compare Closures` option on the input page after the knowledge base and query are submitted. The comparison is structured as a five-step walkthrough moving from a shared BaseRank to the final entailment results so that the differences between the methods are introduced incrementally.

Step 1 presents the shared BaseRank, establishing a common starting point for all three methods. Step 2 introduces where the methods diverge: Rational Closure removes entire ranks while the query antecedent remains exceptional; Lexicographic Closure preserves more information through weakening of the conflicting rank instead of discarding it entirely; and Relevant Closure focuses on the information relevant to the query.

Step 3 allows users to inspect the full step-through of each algorithm individually before returning to the comparison, providing direct access to the reasoning process behind each method rather than relying on the comparison summary. Step 4 presents a visual comparison of how formulas in the shared ranking are treated by each entailment method, including whether they are retained, weakened, or removed. This makes differences in information preservation directly visible and is particularly useful for illustrating the drowning problem, where Rational Closure may discard statements that the bolder entailment methods preserve.

Finally, Step 5 presents the entailment result produced by each method and indicates whether the methods agree. This walkthrough therefore connects the differences observed in the intermediate reasoning to their effect on the entailment result, rather than comparing only the final entailment results.

4.6 Shared Usability Features

Several shared group contributions support the broader PDR system. These include step controls for navigating through the reasoning process, execution-time tracking for the supported entailment methods, a guided tutorial based on the drowning problem, and an information page explaining the purpose of the tool, notation, the input rules required when constructing knowledge bases and queries, and the glossary. Together, these features support self-paced exploration and provide an infrastructure for the Rational Closure step-through, drowning problem example, and comparison environment presented in this paper.

4.7 Pedagogical Design Decisions

The central design decision was the adoption of a debugger-style step-through interface. Existing tools such as EXTRC [7, 19] and the work of Hamayobe [11] provide detailed reasoning results, but do not expose the algorithms' execution. This motivated the exploration of interface designs that could make the reasoning process more visible. An earlier single-page design was rejected because, although simpler, it would have shown the completed BaseRank and entailment result without making the intermediate reasoning process visible. Such a design would limit the pedagogical depth, as users can see what the algorithm produces but cannot follow how or why certain decisions were made during execution. The step-through interface was therefore selected to present one computational stage at a time, allowing users to control the pace of exploration.

An early-stage command-line prototype demonstrated that the BaseRank and Rational Closure traces could be presented sequentially, but the textual presentation made changes to the ranking and working set difficult to visualise. While this validated the underlying trace-capture design, it motivated the development of a web-based debugger-style interface in which these state changes could be represented visually.

An AI-based chatbot was also considered for providing additional explanations, but this was not adopted in the final design because generated explanations are non-deterministic and would require separate validation for correctness and pedagogical consistency. The final system instead uses deterministic explanations derived directly from the recorded algorithm trace, ensuring that the explanations are closely aligned with the actual execution of the algorithms while maintaining a predictable learning experience.

Collectively, these design decisions prioritise pedagogical transparency over implementation simplicity, supporting the aim of making the computational steps of defeasible reasoning visible and inspectable.

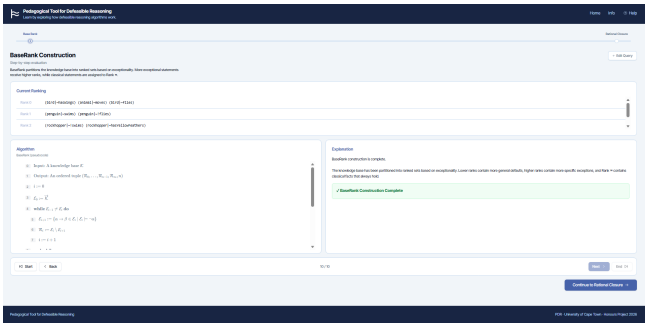


Figure 3: BaseRank step-through interface illustrating the shared debugger-style layout used across the reasoning methods.

Figure 3 illustrates the shared debugger-style step-through layout used across the reasoning methods, including navigation controls and visualisation of the current algorithm state.

4.8 Pedagogical Interface Features

To complement the debugger-style step-through approach, the interface presents formal pseudocode alongside plain-English explanations. The current pseudocode line is highlighted as the corresponding operation is performed, allowing users to relate the formal algorithm to its execution on a knowledge base.

Additional reasoning detail is provided only when required. During Rational Closure exceptionality checks, the `Show Details` option allows users to inspect a justification consisting of a minimal subset of the current materialised working set sufficient to support the exceptionality result [23]. A weak justification is also shown for the final entailment result. This provides access to the reasoning behind individual checks, while keeping the main step-through interface uncluttered.

The tool also uses progressive visualisation to make changes to the algorithm state visible. The BaseRank ranking is populated as statements are assigned to ranks, while Rational Closure visualisation marks ranks as they are removed. The dedicated drowning problem example further allows users to observe how whole-rank removal can discard unrelated information, motivating the need for bolder entailment methods such as Lexicographic and Relevant Closure [3, 4].

The interface design choices are consistent with Nielsen's usability principles, particularly visibility of system status and recognition rather than recall [20]. Consistent terminology, step highlighting, progressive visualisation, colour coding, and separation between the input and walkthrough views further support readability and reduce the amount of information users must track mentally.

5 Results, Analysis, and Discussion

The tool is evaluated on four areas: correctness, pedagogical support, comparative behaviour, and execution time. Correctness is evaluated through manual verification, automated unit testing, and comparison with existing tools. The test cases include strict-only knowledge bases, defeasible-only knowledge bases, multi-rank knowledge bases, entailments, non-entailments, and the drowning problem. The pedagogical analysis examines how closely the step-through interface aligns with the theoretical algorithms, what additional reasoning information it exposes compared with existing tools, and informal expert feedback on the clarity of the interface. Comparative behaviour is examined by applying Rational Closure, Lexicographic Closure, and Relevant Closure to the same knowledge bases and queries to identify where their reasoning processes and entailment results differ. Finally, execution-time observations are used to provide an indicative view of how processing time changes as the knowledge base size increases. A formal user study was not conducted, as the ethical approval, participant recruitment, study execution, and analysis required would have introduced additional work beyond the scope of the project. Instead, informal feedback was gathered from members of the AI research group at the University of Cape Town, providing informal expert observations on the clarity and accessibility of the step-through interface.

5.1 Correctness Evaluation

Manual Verification. Expected outputs were computed by hand using the theoretical algorithms [5] and compared against the

tool's output across a set of test cases covering entailments, non-entailments, multi-rank knowledge bases, and other relevant edge cases for both the BaseRank and Rational Closure algorithms. Tables 2 and 3 in the Appendix present the results. All test cases produced the expected outputs, providing evidence that the BaseRank and Rational Closure implementations behave consistently with the theoretical algorithms across the tested scenarios. For selected test cases, the intermediate trace states were also manually inspected to verify that exceptionality checks, rank assignments, working-set updates, and rank removals occurred in the same sequence prescribed by the theoretical algorithms.

Automated Unit Tests. Automated unit tests were also used to verify the key aspects of the Rational Closure implementation, including entailment and non-entailment behaviour, rank removal in the drowning problem, and the generation of exceptionality and weak justifications. These tests provided additional implementation-level verification that the algorithm and explanation mechanisms behaved as expected.

Comparison with Existing Tools. The same knowledge bases and queries were evaluated using EXTRC [7, 19] and the implementation by Hamayobe [11], and their outputs were compared with those produced by the PDR tool. Table 4 in the Appendix presents the results. The outputs of the PDR tool are consistent with those produced by both of the existing tools across all test cases. Together, these results provide evidence that the BaseRank and Rational Closure implementations behave consistently with the theoretical algorithms across tested scenarios, both in their final outputs and in the intermediate reasoning steps exposed by the interface. This provides a foundation for the pedagogical analysis, since the verified trace behaviour supports the claim that the step-through explanations reflect the corresponding algorithmic states.

5.2 Pedagogical Analysis

The pedagogical support provided by this tool is analysed by examining the alignment of the step-through interface with the theoretical algorithms, comparing the reasoning information exposed by PDR with existing tools, and considering informal expert feedback on the clarity of the interface.

Alignment with Theoretical Algorithms. The step-through interface follows the theoretical algorithms of Casini and Meyer [5] as closely as possible, exposing each step of the BaseRank construction and the Rational Closure entailment process in the same order defined by the algorithms. The correctness evaluation in Section 5.1 provides more evidence of this alignment, showing that both the final outputs and the intermediate reasoning states are consistent with the theoretical algorithms across the tested scenarios. The presentation of formal pseudocode alongside plain-English explanations further supports the pedagogical goal of helping users connect the formal definition of the algorithm with its execution on a concrete knowledge base. By highlighting the current pseudocode line as the corresponding operation is performed, the interface makes the relationship between the formal procedure and the changing algorithm state directly inspectable.

Comparison with Existing Tools. Existing tools such as EXTRC [7, 19] and the implementation by Hamayobe [11] provide reasoning results and explanations, but differ from PDR in how this information is presented. In particular, PDR exposes the reasoning process incrementally through a debugger-style step-through interface. Users can observe the BaseRank being constructed progressively, see the exceptionality checks performed at each iteration, and follow ranks as they are removed during Rational Closure. The `Show Details` functionality further exposes a justification for individual exceptionality checks by identifying a minimal subset of the current materialised working set sufficient to support the exceptionality result. At the final entailment check, the interface also presents a weak justification showing the formulas that contributed to the final result. This changes the interaction from examining a completed explanation to following the sequence of intermediate reasoning steps that produces the result. This supports the pedagogical goal of helping users understand how an entailment conclusion is derived, rather than only observing the completed result.

In addition, PDR provides a dedicated comparison environment in which Rational Closure, Lexicographic Closure, and Relevant Closure are applied to the same knowledge base and query, allowing users to observe where the methods begin to diverge and how those differences affect the final entailment result. The tool also includes a dedicated drowning problem example and explanation, making a known limitation of Rational Closure visible within the reasoning process rather than leaving it as an abstract theoretical limitation. Together, these features support the pedagogical goal of helping users compare the final entailment results, understand the reasoning behaviours that produce them, and observe how the limitations of Rational Closure arise in practice.

Informal Expert Feedback. Informal feedback was received from members of the UCT AI research group who were invited to explore the deployed PDR tool. The feedback was generally positive regarding the clarity of the explanations and the step-by-step presentation of the reasoning process. One reviewer reported that the explanations were clear and did not identify any issues with the reasoning, while another highlighted the step-by-step approach as a strength of the tool. The same reviewer also identified a limitation in the original `Show Details` functionality, where formulas selected as relevant to an exceptionality check did not always correspond to the formulas that logically caused the exceptionality result. This feedback led to the feature being revised to compute classical justifications, adapted from Wang [23], so that the interface could accurately show the formulas responsible for the exceptionality result. This revision also reinforces the pedagogical requirement that explanations presented to users should accurately represent the underlying reasoning process.

5.3 Comparative Behaviour of Entailment Methods

Table 5 compares the behaviour of Rational Closure, Lexicographic Closure, and Relevant Closure across three representative knowledge bases. The selected cases were chosen to show both agreement between the methods and cases where their different treatment of ranked information becomes visible. In the agreement case, all three methods entail $penguin \mid\sim \neg flies$. Although the final result is the same, the methods do not reach it in the same way. This shows that agreement on the entailment result does not imply the same treatment of the ranked knowledge base.

The clearest divergence occurs in the drowning problem. Rational Closure does not entail *penguin* $\nvdash$ *wings* because the entire Rank 0 is removed once the query antecedent is found to be exceptional, causing the statement *bird* $\nvdash$ *wings*, which is unrelated to the exception, to be discarded together with the conflicting information. Lexicographic Closure preserves more information by weakening the conflicting rank, while Relevant Closure retains sufficient relevant information to derive the query. As a result, the bolder entailment methods derive the conclusion that Rational Closure loses through whole-rank removal.

The additional multi-rank case shows that differences between the methods are not limited to cases where their final entailment results disagree. All three methods entail *rockhopper* $\nvdash \neg$*swims*, but they reach this result using different transformations of the ranked knowledge base. Rational Closure removes Rank 0 and Rank 1 completely, Lexicographic Closure removes Rank 0 while weakening Rank 1, and Relevant Closure operates only on the information relevant to the query. This demonstrates that the comparison environment can expose differences in the reasoning process even when the final entailment result is identical.

5.4 Execution-Time Observation

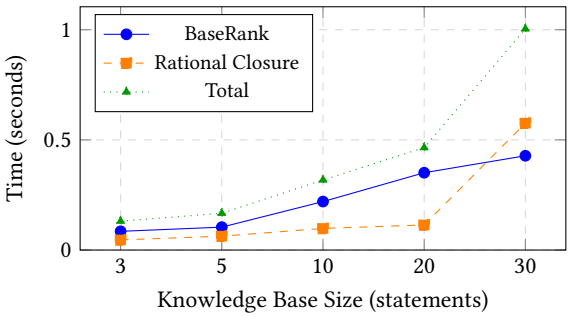


Figure 4: Average execution times for Rational Closure across increasing knowledge-base sizes.

Figure 4 shows the average execution time observed as the size of the knowledge base increased. Overall execution time increased across the tested knowledge bases, from 0.131 seconds for three statements to 1.004 seconds for 30 statements. BaseRank construction showed a gradual increase across the tested sizes, while Rational Closure execution remained comparatively low up to 20 statements before increasing substantially for the 30-statement knowledge base. This suggests that execution time is influenced not only by knowledge-base size, but also by its structure and the number of exceptionality and entailment checks required for a query. The full timing results are provided in Appendix C.

These measurements are intended as an indicative observation rather than a formal performance benchmark. The implementation prioritises pedagogical transparency through detailed trace capture rather than computational optimisation. The results therefore support the identified trade-off between exposing detailed reasoning states and execution efficiency, particularly as knowledge bases become more complex.

5.5 Discussion

The results provide evidence that the PDR tool exposes computational states that are consistent with the BaseRank and Rational Closure algorithms, addressing the research question in Section 1. The correctness evaluation, including automated unit tests, manual verification, and comparison with existing implementations, showed that both the final outputs and selected reasoning states were consistent with the theoretical algorithms [5] and with the outputs of existing implementations [7, 11, 19]. This is important for the pedagogical purpose of the tool, as the explanations and visualisations presented to users depend on the underlying reasoning process being represented correctly.

The pedagogical analysis further indicates that PDR provides greater visibility into the intermediate reasoning process than the compared tools. Instead of presenting the reasoning information as a completed result or explanation, the debugger-style interface allows users to follow the progression from the BaseRank construction through exceptionality checking and rank removal to the final entailment decision. This makes the relationship between the formal algorithm and its execution more explicit, while features such as progressive visualisation and optional detail allow the reasoning process to be inspected incrementally.

The informal feedback supports the observations made in the pedagogical analysis, with reviewers highlighting the clarity of the explanations and the usefulness of the step-by-step presentation. Importantly, the feedback highlighted the need to ensure that the information shown accurately reflects the underlying reasoning process. However, neither this feedback nor the evaluation establishes that the interface improves learning outcomes or user comprehension, as no formal user study was conducted.

An important finding is that the drowning problem, a known limitation of Rational Closure, can be demonstrated concretely through the predefined example, allowing users to observe not only what the algorithm computes, but also how its limitations arise within the reasoning process.

The comparative results also show that differences between defeasible entailment methods are not always visible from the final entailment result alone. In the multi-rank example, Rational Closure, Lexicographic Closure, and Relevant Closure all derive the same conclusion despite treating the ranked knowledge base differently. The comparison environment therefore exposes the differences in the intermediate reasoning process that would otherwise be hidden by identical final results. In the drowning problem example, these differences affect the final entailment result: Rational Closure removes the whole exceptional rank and loses the unrelated *bird* $\nvdash$ *wings* statement, whereas the bolder entailment methods preserve sufficient information to derive the query. This provides a concrete illustration of why Lexicographic Closure and Relevant Closure were introduced as alternatives to Rational Closure [3, 4].

The execution-time observations further highlight the trade-off between pedagogical transparency and computational efficiency. Although the execution time remained below approximately one second for the tested knowledge bases, the increase in processing time at larger sizes suggests that further optimisations may be required for larger or more complex knowledge bases.

Several directions identified as future work by Hamayobe [11] have been addressed in this tool, including interactive visualisation of the reasoning process, transparent statement assignment and removal, and plain-English explanations alongside formal derivations. This extends prior work and provides a foundation for further pedagogical and algorithmic extensions.

These findings suggest that exposing intermediate reasoning states provides a useful mechanism for supporting the interpretation of defeasible reasoning algorithms. Overall, the results provide evidence that the tool exposes the relevant computational states and provides additional opportunities for users to inspect, compare, and interpret the reasoning process. The approach could therefore provide a foundation for future user studies and for extending step-through visualisations to additional defeasible reasoning algorithms.

5.6 Limitations

A formal user study was not conducted, as designing and carrying out such a study would require additional ethical approval, participant recruitment, data collection, and analysis beyond the scope of the project. Consequently, the effect of the tool on learning outcomes or user comprehension cannot be formally established.

The tool prioritises pedagogical transparency over computational efficiency by following the theoretical algorithm as closely as possible and capturing detailed trace information at each step. The execution-time observations in Section 5.4 indicate that processing time increases as the knowledge base becomes larger and more complex, suggesting that further optimisation may be required for substantially larger knowledge bases.

The drowning problem is demonstrated through a predefined example rather than automatic detection. Implementing reliable detection would require additional SAT-solving operations and was therefore left outside the scope of the project.

Finally, users must enter knowledge bases and queries using the supported logical notation, as the tool does not provide natural-language input or translation functionality. This may increase the initial learning barrier for users who are unfamiliar with defeasible logic syntax.

6 Conclusion

This paper presented a web-based pedagogical step-through tool for defeasible reasoning, focusing on Rational Closure. The tool exposes the BaseRank construction and Rational Closure entailment process step-by-step, presenting intermediate computational states alongside plain-English explanations and formal pseudocode. It forms part of a broader collaborative system, which also includes Lexicographic and Relevant Closure, implemented by other group members.

The main contributions of this paper are a Rational Closure implementation with full trace capture and a debugger-style step-through interface that combines pseudocode, plain-English explanations, and progressive rank visualisations; justification-based explanations for exceptionality checks and the final entailment check; a dedicated drowning problem example illustrating a known limitation of Rational Closure; and a comparison environment that allows Rational Closure, Lexicographic Closure, and Relevant Closure to be examined and compared using the same knowledge base and query. Collectively, these features make different aspects of Rational Closure more visible by exposing its intermediate reasoning process, demonstrating how its limitations arise, and showing how its treatment of ranked information differs from other defeasible entailment methods. In this way, they address the research question in Section 1 by supporting the inspection and interpretation of Rational Closure while also enabling comparison with alternative entailment methods.

The evaluation provides evidence that the tested implementations behave consistently with the theoretical algorithms, while the comparison and drowning problem examples demonstrate how the tool can expose differences and limitations that are not visible from the final entailment results alone. Informal expert feedback also highlighted the importance of ensuring that the explanations shown by the interface accurately reflect the underlying reasoning process. This feedback led to the `Show Details` functionality being revised to use justification-based explanations. Although the effect on learning outcomes was not formally evaluated, the tool provides a foundation for future pedagogical studies and extensions.

6.1 Future Work

Several directions for future work have been identified. A formal user study could be conducted to measure the pedagogical effectiveness of the tool, complementing the informal evaluation presented here. Performance optimisations, including cached entailment strategies similar to those implemented in EXTRC [7, 19], together with further scaling experiments on larger knowledge bases, could provide a more comprehensive evaluation of performance. Automatic detection of the drowning problem would strengthen the tool's explanatory capabilities beyond the predefined example currently provided. Finally, the integration of a natural language generator, developed as part of the broader EXTRC project [7], could allow defeasible and classical statements to be automatically translated into readable natural language phrases. For example, $bird \mid\sim flies$ could be expressed as "birds typically fly", while $penguin \rightarrow bird$ could be expressed as "penguins are always birds". This could improve accessibility by allowing users to focus more on the reasoning process rather than on interpreting the symbolic representation.

AI Use Declaration

ChatGPT (OpenAI) and Claude (Anthropic) were used to assist with proofreading and refinement of the paper, including feedback on grammar, phrasing, flow, and presentation of the technical content. They were also used during software development to assist with frontend UI components, debugging, and resolving implementation issues. All AI suggestions were reviewed and verified. The individual research contributions and design decisions mentioned in the paper are the author's own work, while shared, adapted, and literature-derived content is identified throughout the paper.

References

[1] Mordechai Ben-Ari. 2012. *Mathematical Logic for Computer Science* (3rd ed.). Springer Publishing Company, Incorporated. doi:10.1007/978-1-4471-4129-7

[2] Ronald J Brachman, Hector J Levesque, and Maurice Pagnucco. 2004. *Knowledge representation and reasoning*. Vol. 1. Elsevier. doi:10.1016/B978-1-55860-932-

7.X5083-3

[3] Giovanni Casini, Thomas Meyer, Kodylan Moodley, and Riku Nortjé. 2014. Relevant Closure: A New Form of Defeasible Reasoning for Description Logics. In *Logics in Artificial Intelligence*, Eduardo Fermé and João Leite (Eds.). Springer International Publishing, Cham, 92–106. https://doi.org/10.1007/978-3-319-11558-0_7

[4] Giovanni Casini, Thomas Meyer, and Ivan Varzinczak. 2018. Defeasible Entailment: from Rational Closure to Lexicographic Closure and Beyond. In *Proceedings of the 17th International Workshop on Non-Monotonic Reasoning (NMR 2018)*. Tempe, Arizona, USA. https://orbilu.uni.lu/handle/10993/37393

[5] Giovanni Casini, Thomas Meyer, and Ivan Varzinczak. 2019. Taking Defeasible Entailment Beyond Rational Closure. In *Logics in Artificial Intelligence*, Francesco Calimeri, Nicola Leone, and Marco Manna (Eds.). Springer International Publishing, Cham, 182–197. doi:10.1007/978-3-030-19570-0_12

[6] Giovanni Casini and Umberto Straccia. 2010. Rational Closure for Defeasible Description Logics. In *Logics in Artificial Intelligence*, Tomi Janhunen and Ilkka Niemelä (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 77–90. doi:10.1007/978-3-642-15675-5_9

[7] Julia Cotterrell. 2025. A User Interface to Aid in the Understanding of Rational Closure Entailments. https://projects.cs.uct.ac.za/honsproj/ Honours Project, University of Cape Town.

[8] Michael Freund. 1998. Preferential reasoning in the perspective of Poole default logic. *Artificial Intelligence* 98, 1 (1998), 209–235. doi:10.1016/S0004-3702(97)00053-2

[9] Jean H Gallier. 2015. *Logic for computer science: foundations of automatic theorem proving*. Courier Dover Publications.

[10] Gaetano Geck, Artur Ljulin, Sebastian Peter, Jonas Schmidt, Fabian Vehlken, and Thomas Zeume. 2018. Introduction to Iltis: an interactive, web-based system for teaching logic. In *Proceedings of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education* (Larnaca, Cyprus) *(ITiCSE 2018)*. Association for Computing Machinery, New York, NY, USA, 141–146. doi:10.1145/3197091.3197095

[11] Chipo Hamayobe. 2026. *Implementation and Evaluation of KLM-Style Defeasible Entailment and Explanation Algorithms*. Master of Science Dissertation. University of Cape Town, Cape Town, South Africa. Supervisors: Thomas Meyer and Jonas Philipp Haldiman.

[12] Matthew Horridge. 2011. *Justification Based Explanation in Ontologies*. Ph. D. Dissertation. The University of Manchester. https://www.research.manchester.ac.uk/portal/en/theses/justification-based-explanation-in-ontologies(7a9d7700-e12f-43be-93b3-c79966f3a521).html

[13] Amogh Inamdar, Uzay Macar, Michel Vazirani, Michael Tarnow, Zarina Mustapha, Natalia Dittren, Sam Sadeh, Nakul Verma, and Ansaf Salleb-Aouissi. 2025. LogicLearner: A Tool for the Guided Practice of Propositional Logic Proofs. arXiv:2503.19280 [cs.DM] https://arxiv.org/abs/2503.19280

[14] A. Kaliski. 2020. *An Overview of KLM-Style Defeasible Entailment*. Master's thesis. University of Cape Town, Cape Town, South Africa. http://hdl.handle.net/11427/32743

[15] Sarit Kraus, Daniel Lehmann, and Menachem Magidor. 1990. Nonmonotonic reasoning, preferential models and cumulative logics. *Artificial Intelligence* 44, 1 (1990), 167–207. doi:10.1016/0004-3702(90)90101-5

[16] Daniel Lehmann. 1995. Another Perspective on Default Reasoning. *Annals of Mathematics and Artificial Intelligence* 15, 1 (March 1995), 61–82. doi:10.1007/BF01535841

[17] Daniel Lehmann and Menachem Magidor. 1992. What Does a Conditional Knowledge Base Entail? *Artificial Intelligence* 55, 1 (1992), 1–60. doi:10.1016/0004-3702(92)90041-u

[18] Hector J. Levesque. 1986. Knowledge Representation and Reasoning. *Annual Review of Computer Science* 1 (1986), 255–287. doi:10.1146/annurev.cs.01.060186.001351

[19] Thabo Moloi. 2024. Extending Defeasible Reasoning Beyond Rational Closure. https://projects.cs.uct.ac.za/honsproj/ Honours Project, University of Cape Town.

[20] Jakob Nielsen and Rolf Molich. 1990. Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Seattle, Washington, USA) *(CHI '90)*. Association for Computing Machinery, New York, NY, USA, 249–256. doi:10.1145/97243.97281

[21] John L. Pollock. 1987. Defeasible reasoning. *Cognitive Science* 11, 4 (1987), 481–518. doi:10.1016/S0364-0213(87)80017-4

[22] Matthias Thimm. 2017. The Tweety Library Collection for Logical Aspects of Artificial Intelligence and Knowledge Representation. *KI - Künstliche Intelligenz* 31, 1 (March 2017), 93–97. doi:10.1007/s13218-016-0458-4

[23] Shun Wang. 2023. *Defeasible justification for the KLM Framework*. Master of Science Dissertation. University of Cape Town, Cape Town, South Africa. http://hdl.handle.net/11427/39922

A System Architecture

A.1 System Architecture Diagram

Figure 5 presents a larger version of the system architecture shown in the main paper, providing a clearer view of the separation between the frontend, backend, and communication between them.

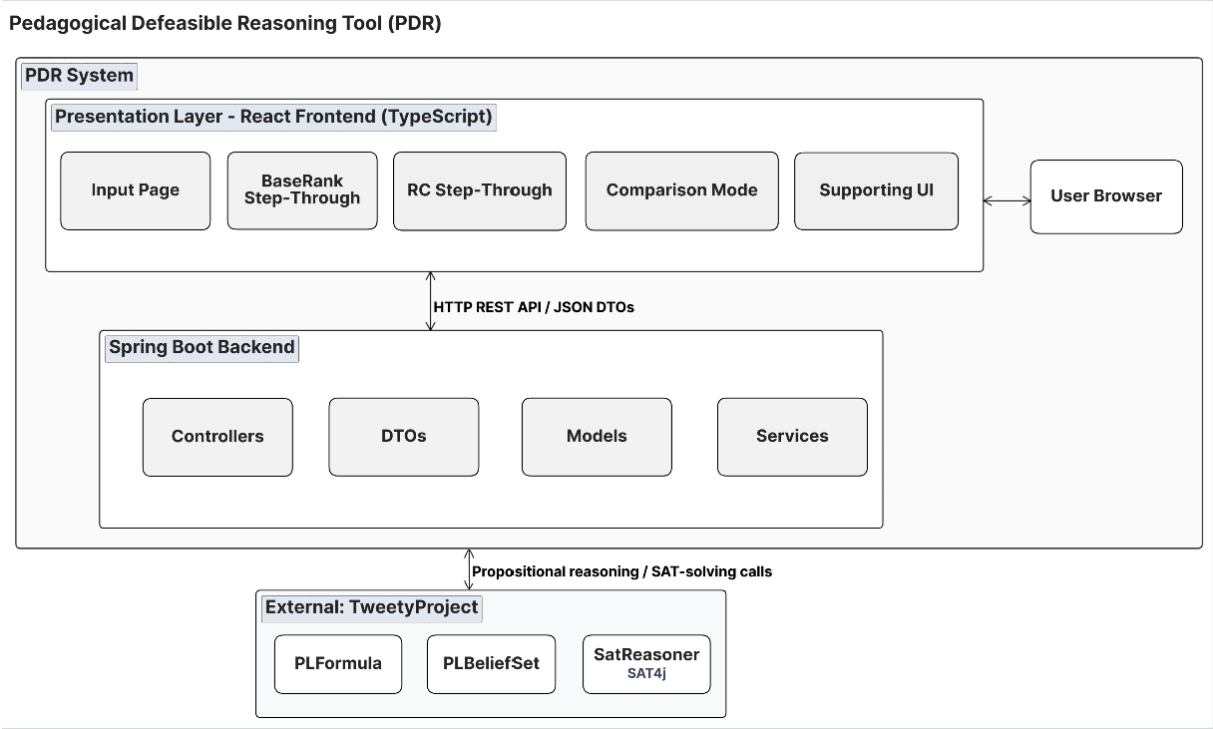


Figure 5: Larger view of the System Architecture of the Pedagogical Defeasible Reasoning Tool

A.2 Backend Implementation

The backend is implemented in Java using the Spring Boot framework and follows a layered architecture consisting of controller, service, and model layers. The service layer contains the reasoning functionality, while the model layer represents the knowledge bases, rankings, entailment results, and the intermediate trace information exposed by the tool. Logical inference is performed using the TweetyProject library [22], which provides propositional reasoning and SAT-solving functionality used throughout BaseRank and the defeasible entailment algorithms.

The internal knowledge-base representation is adapted from EXTRC [7, 19] and built using TweetyProject's propositional logic classes [22]. Incoming knowledge bases are separated into classical and defeasible statements, with defeasible statements materialised into classical implications where required for satisfiability and exceptionality checking. The backend is designed to be modular and extensible, following a similar architectural approach to prior implementations [7, 11, 19], allowing future integration of additional reasoning algorithms.

The backend supports multiple entailment methods through a common reasoning interface. A Strategy pattern allows each reasoner to implement a shared entailment interface independently, while a Factory pattern instantiates the appropriate reasoner according to the user's chosen algorithm. This allows Rational Closure, Lexicographic Closure, and Relevant Closure to operate through a consistent workflow within the same system, while avoiding tight coupling between the frontend and any specific entailment method and enabling the comparison environment.

A.3 Frontend Implementation

The frontend is implemented using React with TypeScript, chosen for its component-based architecture which naturally supports the modular structure of the step-through interface. Its primary role is to transform the reasoning traces returned by the backend into an interactive pedagogical representation. Users can progress through the reasoning process one step at a time while the interface updates the ranking visualisation, highlights the corresponding pseudocode, and presents plain-English explanations of the current operation. The frontend therefore focuses on presenting and explaining the reasoning process, while the backend is responsible for the underlying computation.

A.4 API Communication

Communication between the frontend and backend is handled through a REST API. The knowledge base is submitted first, after which the system computes and stores the BaseRank result. Entailment queries are then evaluated against this ranking, allowing the same BaseRank result to be reused across different reasoning methods, while avoiding unnecessary recomputation. The backend returns the final result and trace information required by the corresponding step-through interface.

B Test Cases

Knowledge Base	Expected Ranking	Actual Ranking	Match
$\{bird \mid\sim flies, penguin \rightarrow bird, penguin \mid\sim \neg flies\}$	• $\mathcal{R}_0 = \{bird \mid\sim flies\}$ • $\mathcal{R}_1 = \{penguin \mid\sim \neg flies\}$ • $\mathcal{R}_\infty = \{penguin \rightarrow bird\}$	• $\mathcal{R}_0 = \{bird \mid\sim flies\}$ • $\mathcal{R}_1 = \{penguin \mid\sim \neg flies\}$ • $\mathcal{R}_\infty = \{penguin \rightarrow bird\}$	Yes
$\{bird \mid\sim flies, bird \mid\sim wings, penguin \rightarrow bird, penguin \mid\sim \neg flies\}$	• $\mathcal{R}_0 = \{bird \mid\sim flies, bird \mid\sim wings\}$ • $\mathcal{R}_1 = \{penguin \mid\sim \neg flies\}$ • $\mathcal{R}_\infty = \{penguin \rightarrow bird\}$	• $\mathcal{R}_0 = \{bird \mid\sim flies, bird \mid\sim wings\}$ • $\mathcal{R}_1 = \{penguin \mid\sim \neg flies\}$ • $\mathcal{R}_\infty = \{penguin \rightarrow bird\}$	Yes
$\{bird \mid\sim flies\}$	• $\mathcal{R}_0 = \{bird \mid\sim flies\}$ • $\mathcal{R}_\infty = \{\}$	• $\mathcal{R}_0 = \{bird \mid\sim flies\}$ • $\mathcal{R}_\infty = \{\}$	Yes
$\{penguin \rightarrow bird\}$	• $\mathcal{R}_\infty = \{penguin \rightarrow bird\}$	• $\mathcal{R}_\infty = \{penguin \rightarrow bird\}$	Yes
$\{animal \mid\sim moves, bird \rightarrow animal, bird \mid\sim flies, bird \mid\sim hasWings, penguin \rightarrow bird, penguin \mid\sim \neg flies, penguin \mid\sim swims, rockhopper \rightarrow penguin, rockhopper \mid\sim hasYellowFeathers, rockhopper \mid\sim \neg swims\}$	• $\mathcal{R}_0 = \{animal \mid\sim moves, bird \mid\sim flies, bird \mid\sim hasWings\}$ • $\mathcal{R}_1 = \{penguin \mid\sim \neg flies, penguin \mid\sim swims\}$ • $\mathcal{R}_2 = \{rockhopper \mid\sim \neg swims, rockhopper \mid\sim hasYellowFeathers\}$ • $\mathcal{R}_\infty = \{penguin \rightarrow bird, rockhopper \rightarrow penguin, bird \rightarrow animal\}$	• $\mathcal{R}_0 = \{animal \mid\sim moves, bird \mid\sim flies, bird \mid\sim hasWings\}$ • $\mathcal{R}_1 = \{penguin \mid\sim \neg flies, penguin \mid\sim swims\}$ • $\mathcal{R}_2 = \{rockhopper \mid\sim \neg swims, rockhopper \mid\sim hasYellowFeathers\}$ • $\mathcal{R}_\infty = \{penguin \rightarrow bird, rockhopper \rightarrow penguin, bird \rightarrow animal\}$	Yes

Table 2: BaseRank correctness test cases

Knowledge Base	Query	Expected	Actual	Match
$\{bird \mid\sim flies\}$	$penguin \mid\sim flies$	NOT ENTAILED	NOT ENTAILED	Yes
$\{penguin \rightarrow bird\}$	$penguin \mid\sim bird$	ENTAILED	ENTAILED	Yes
$\{bird \mid\sim flies, penguin \rightarrow bird, penguin \mid\sim \neg flies\}$	$penguin \mid\sim \neg flies$	ENTAILED	ENTAILED	Yes
$\{bird \mid\sim flies, bird \mid\sim wings, penguin \rightarrow bird, penguin \mid\sim \neg flies\}$	$penguin \mid\sim wings$	NOT ENTAILED	NOT ENTAILED	Yes
$\{animal \mid\sim moves, bird \rightarrow animal, bird \mid\sim flies, bird \mid\sim hasWings, penguin \rightarrow bird, penguin \mid\sim \neg flies, penguin \mid\sim swims, rockhopper \rightarrow penguin, rockhopper \mid\sim hasYellowFeathers, rockhopper \mid\sim \neg swims\}$	$rockhopper \mid\sim \neg swims$	ENTAILED	ENTAILED	Yes

Table 3: Rational Closure correctness test cases

Knowledge Base	Query	Expected	PDR	EXTRC	Hamayobe
$\{bird \mid\sim flies, penguin \rightarrow bird, penguin \mid\sim \neg flies\}$	$penguin \mid\sim \neg flies$	ENTAILED	ENTAILED	ENTAILED	ENTAILED
$\{bird \mid\sim flies, bird \mid\sim wings, penguin \rightarrow bird, penguin \mid\sim \neg flies\}$	$penguin \mid\sim wings$	NOT ENTAILED	NOT ENTAILED	NOT ENTAILED	NOT ENTAILED
$\{cat \mid\sim active, garfield \rightarrow cat, garfield \mid\sim \neg active\}$	$garfield \mid\sim \neg active$	ENTAILED	ENTAILED	ENTAILED	ENTAILED
$\{bird \mid\sim flies\}$	$penguin \mid\sim flies$	NOT ENTAILED	NOT ENTAILED	NOT ENTAILED	NOT ENTAILED
$\{rockhopper\ KB\}$	$rockhopper \mid\sim \neg swims$	ENTAILED	ENTAILED	ENTAILED	ENTAILED

Table 4: Comparison of entailment results across tools

Test Case	Query	Rational Closure	Lexicographic Closure	Relevant Closure	Observed Difference
Case 1: Agreement Case: Penguins KB	$penguin \mid\sim \neg flies$	Entailed, lower exceptional rank removed as required.	Entailed, produces the same final result.	Entailed, produces the same final result.	All three methods agree on the entailment result despite using different reasoning strategies.
Case 2: Drowning Problem KB	$penguin \mid\sim wings$	Not entailed, Rank 0 removed entirely	Entailed, conflicting information in Rank 0 is weakened while compatible information is retained.	Entailed, sufficient information preserved to derive the query.	RC loses the unrelated $bird \mid\sim wings$ statement through whole-rank removal, while the bolder closures preserve sufficient information to derive the query.
Case 3: additional divergence case (multi-ranked KB): Penguins Extended	$rockhopper \mid\sim \neg swims$	Entailed, Rank 0 & Rank 1 are removed	Entailed, Rank 0 removed and Rank 1 weakened	Entailed, only the relevant conflicting information is ignored while other information can remain available.	Although all methods entail the query, they reach the result using different treatments of the ranked knowledge base: RC removes complete exceptional ranks, LC preserves part of Rank 1 through weakening, while Relevant Closure modifies only information relevant to the query.

Table 5: Comparison of the behaviour of supported defeasible entailment methods across representative test cases.

C Execution-Time Results

For each knowledge-base size, the same Rational Closure query was executed five times on the same machine and software environment, and the reported values represent the mean execution time across the five runs. The knowledge bases were based on the same Penguin domain and were constructed to remain structurally comparable as additional statements were introduced, while the query type was kept consistent across tests. No separate warm-up phase was performed. However, execution times generally decreased across repeated runs, indicating that runtime warm-up or caching effects may have influenced the measurements. The results should therefore be interpreted as execution-time observations rather than a formal performance benchmark.

KB size	BaseRank avg.	Rational Closure avg.	Total avg.
3 Statements	0.085s	0.046s	0.131s
5 Statements	0.104s	0.063s	0.167s
10 Statements	0.220s	0.098s	0.318s
20 Statements	0.351s	0.114s	0.465s
30 Statements	0.428s	0.576s	1.004s

Table 6: Average execution times for Rational Closure across increasing knowledge-base sizes.

D Screenshots of UI

This appendix presents selected interface panels from the pedagogical defeasible reasoning tool. Rather than showing a complete walkthrough, the figures illustrate the main interfaces and pedagogical features discussed in the paper, including knowledge base input, BaseRank construction, Rational Closure execution, justification details, the drowning problem explanation, comparison mode, and supporting guidance pages.

D.1 Input and Information

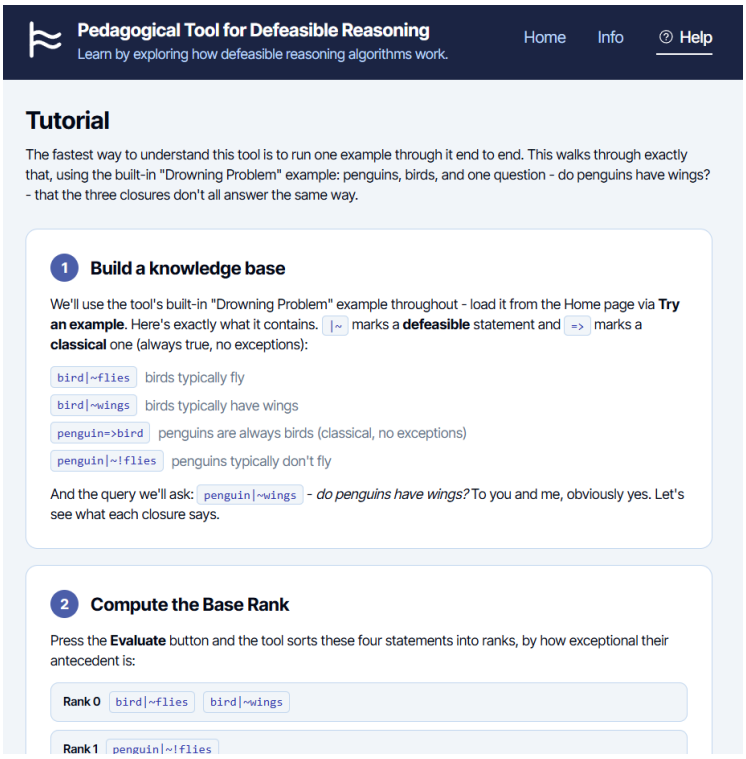


Figure 6: Guided tutorial interface.

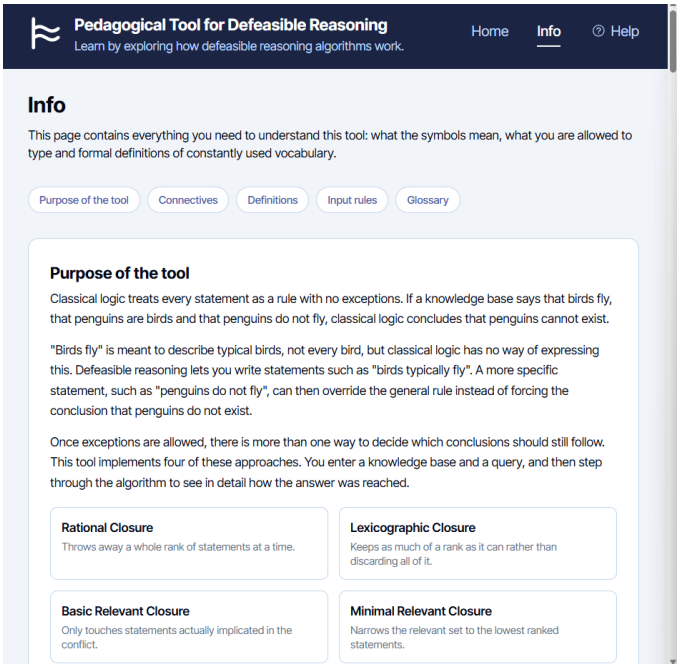


Figure 7: Information page explaining notation and tool usage.

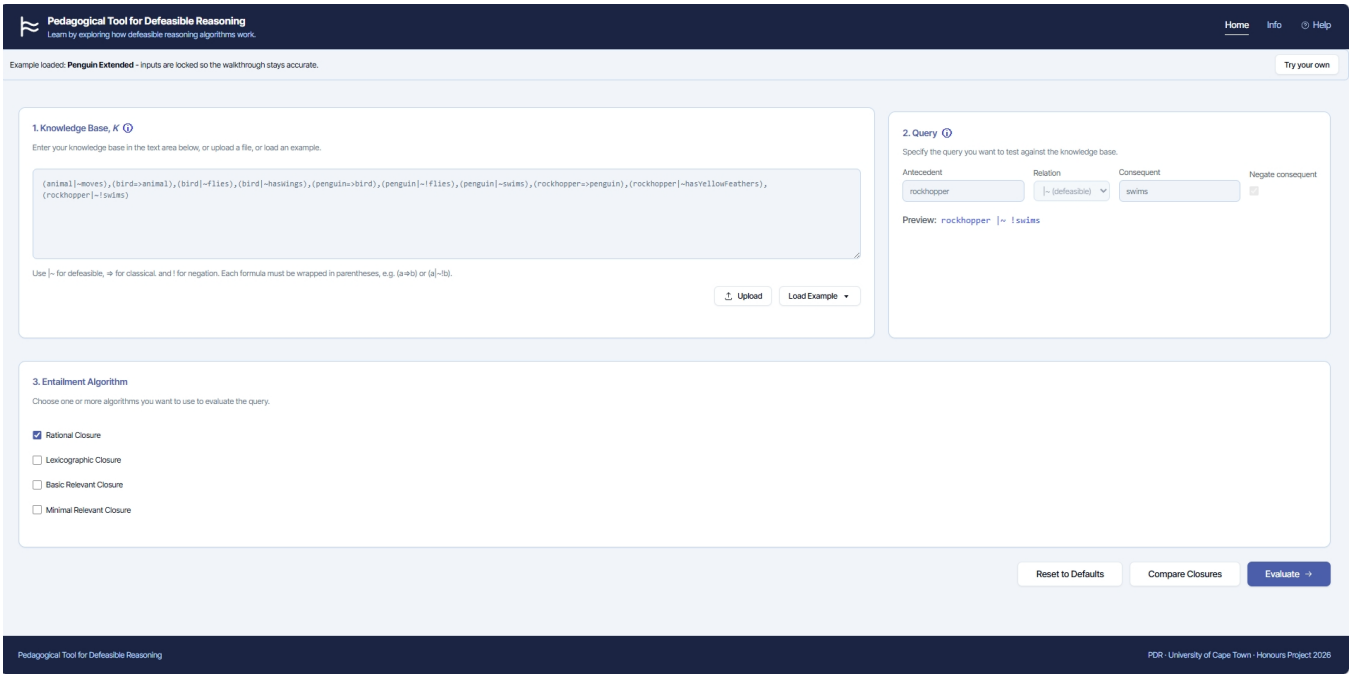


Figure 8: Knowledge-base and query input page.

D.2 BaseRank Visualisation

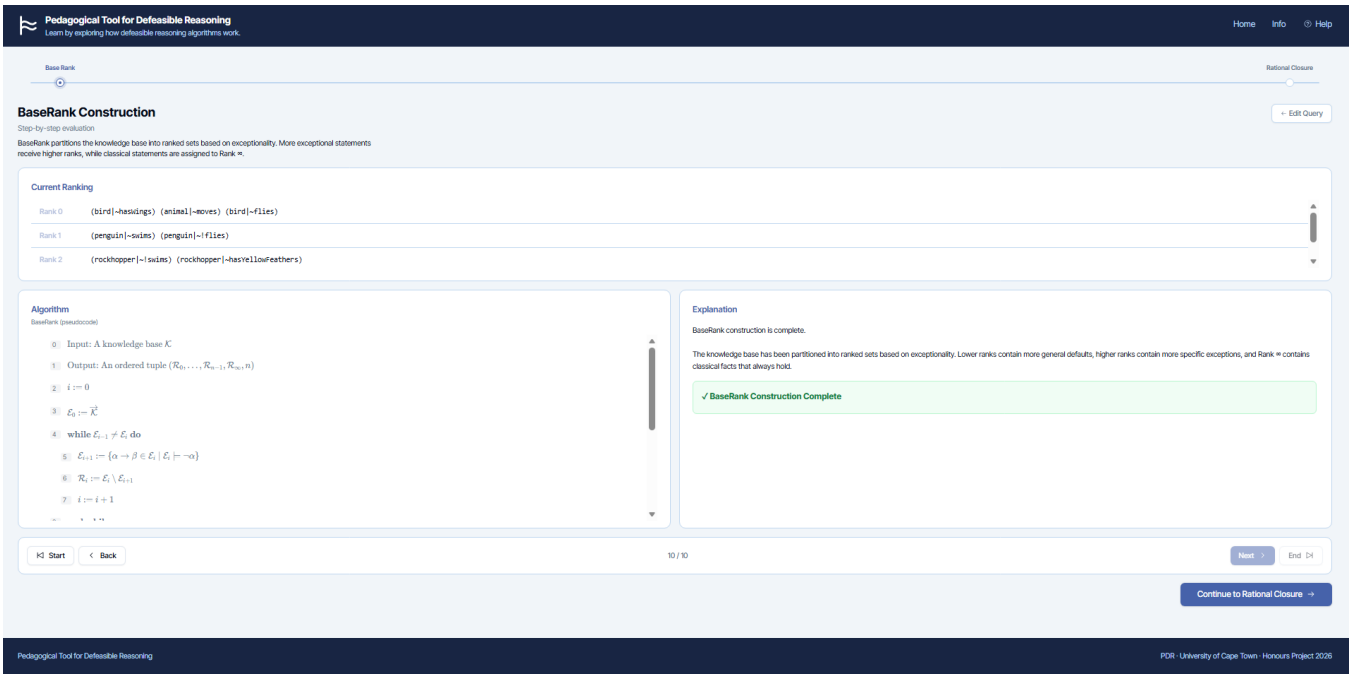


Figure 9: Overall BaseRank step-through interface

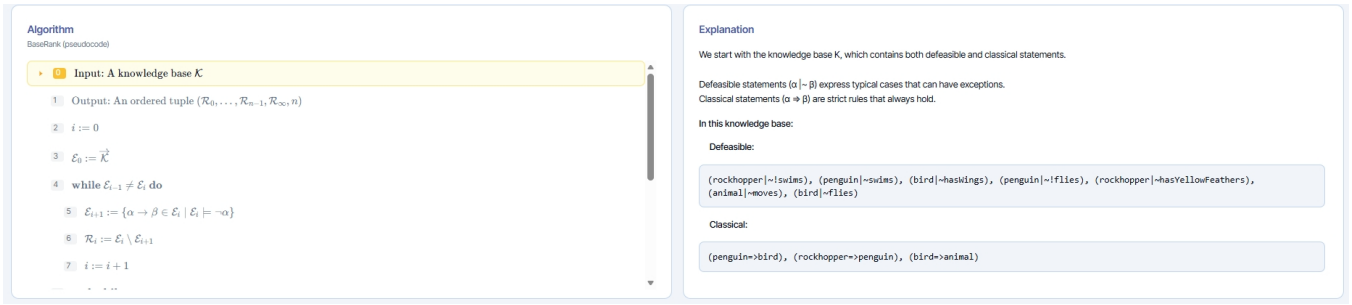


Figure 10: Pseudocode highlighting and plain-English explanation displayed side-to-side during the BaseRank step-through.

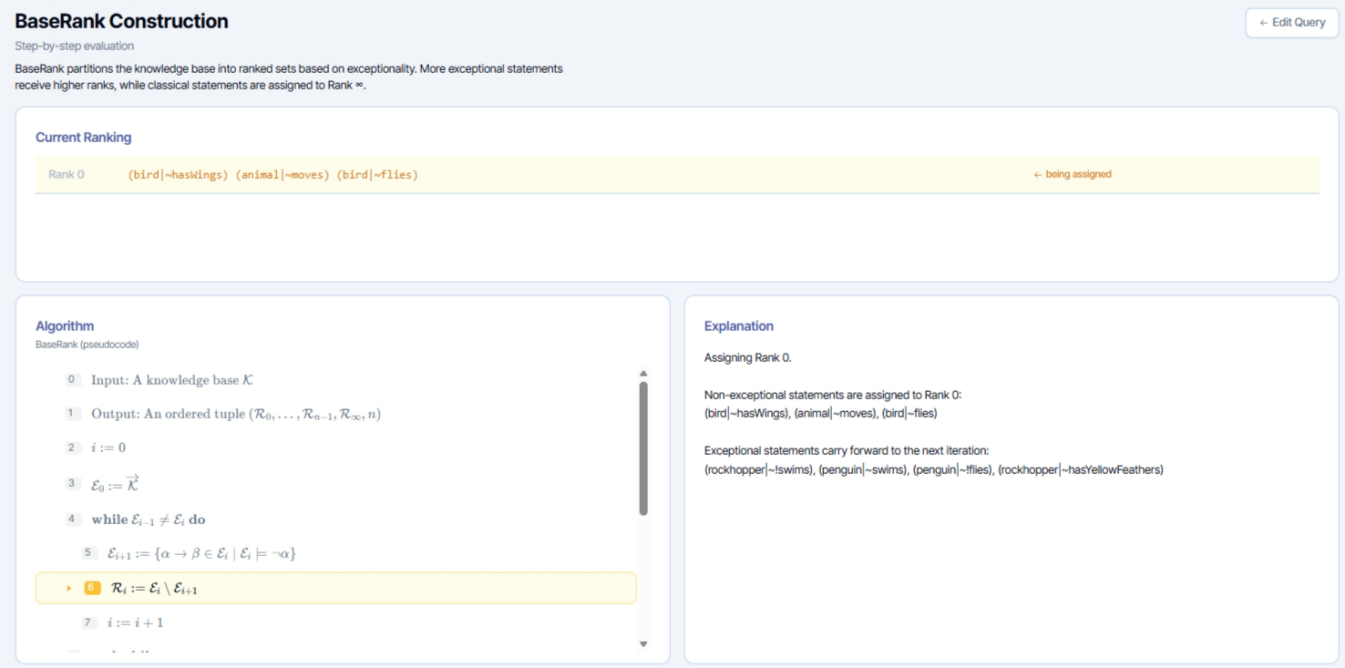


Figure 11: Rank assignment during BaseRank construction.

D.3 Rational Closure Interface

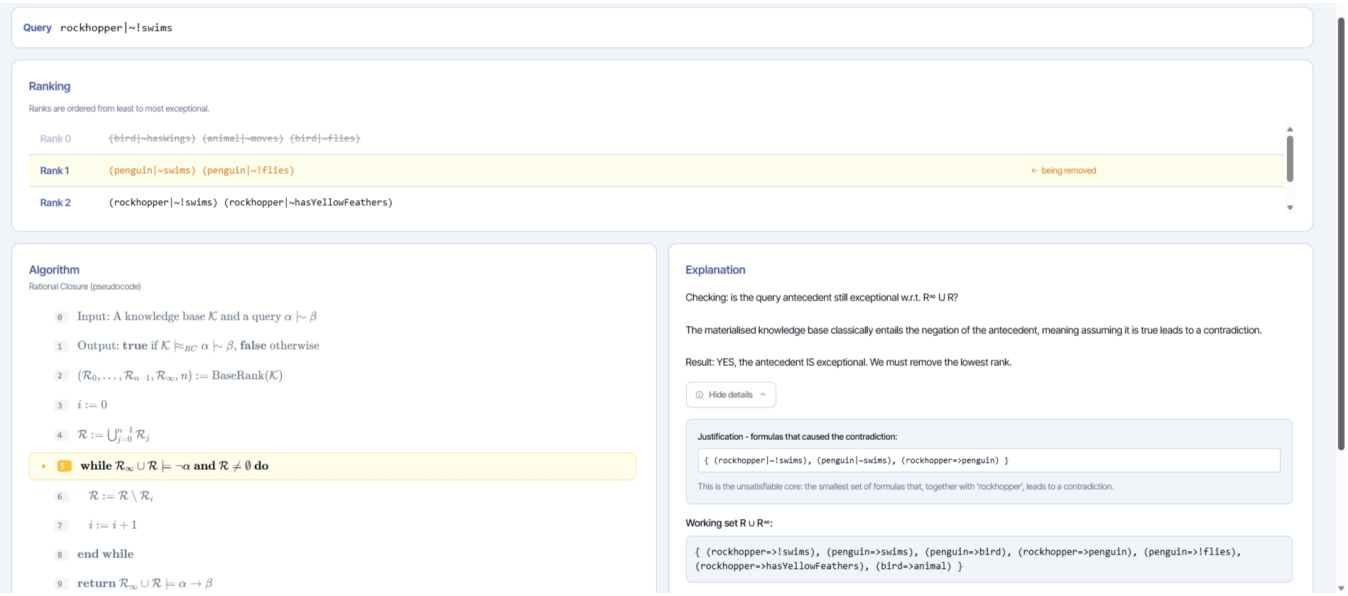


Figure 12: Rank removal during Rational Closure.

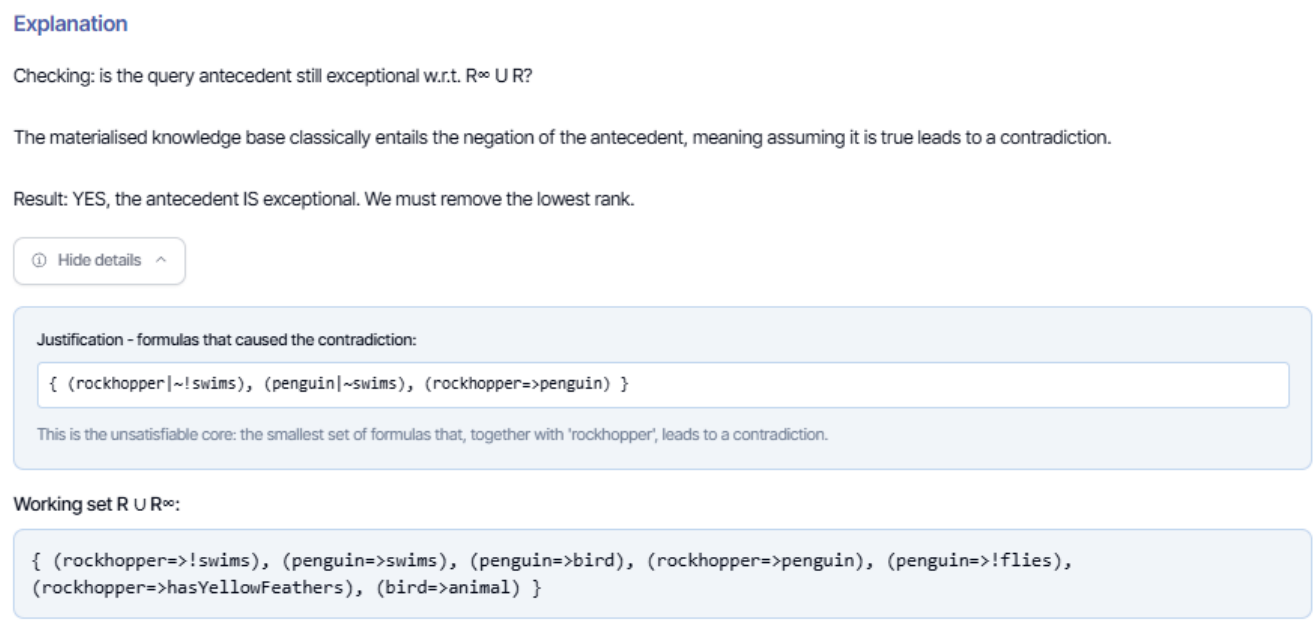


Figure 13: *Show Details option:* Justification for an exceptionality check.

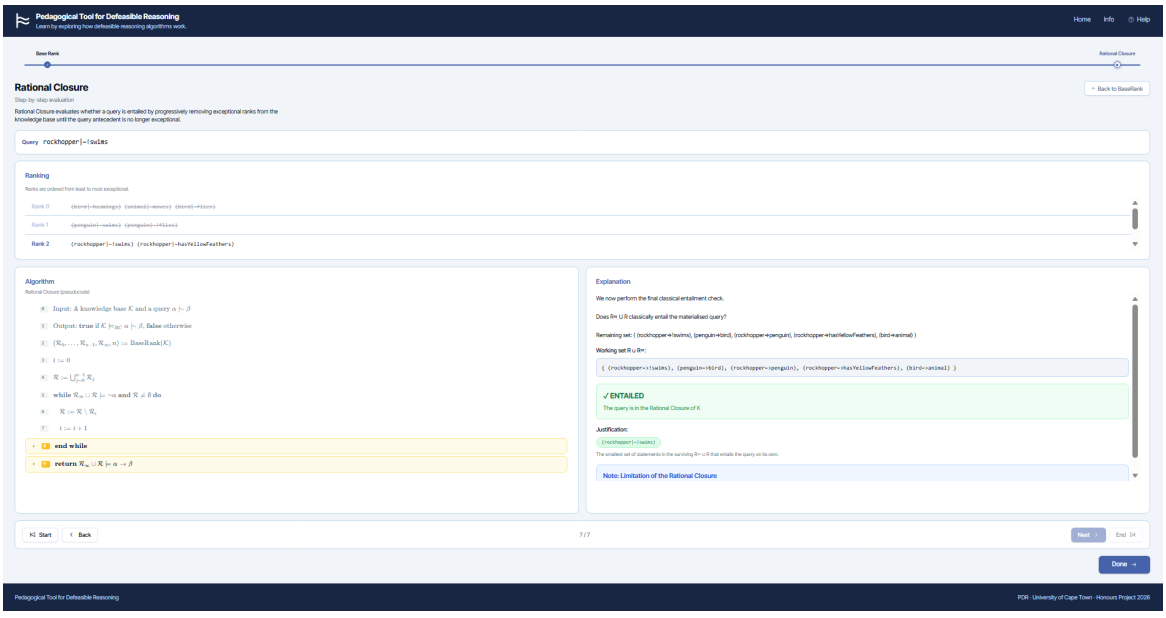


Figure 14: Overall Rational Closure step-through interface.

D.4 Predefined Drowning Problem Visualisation

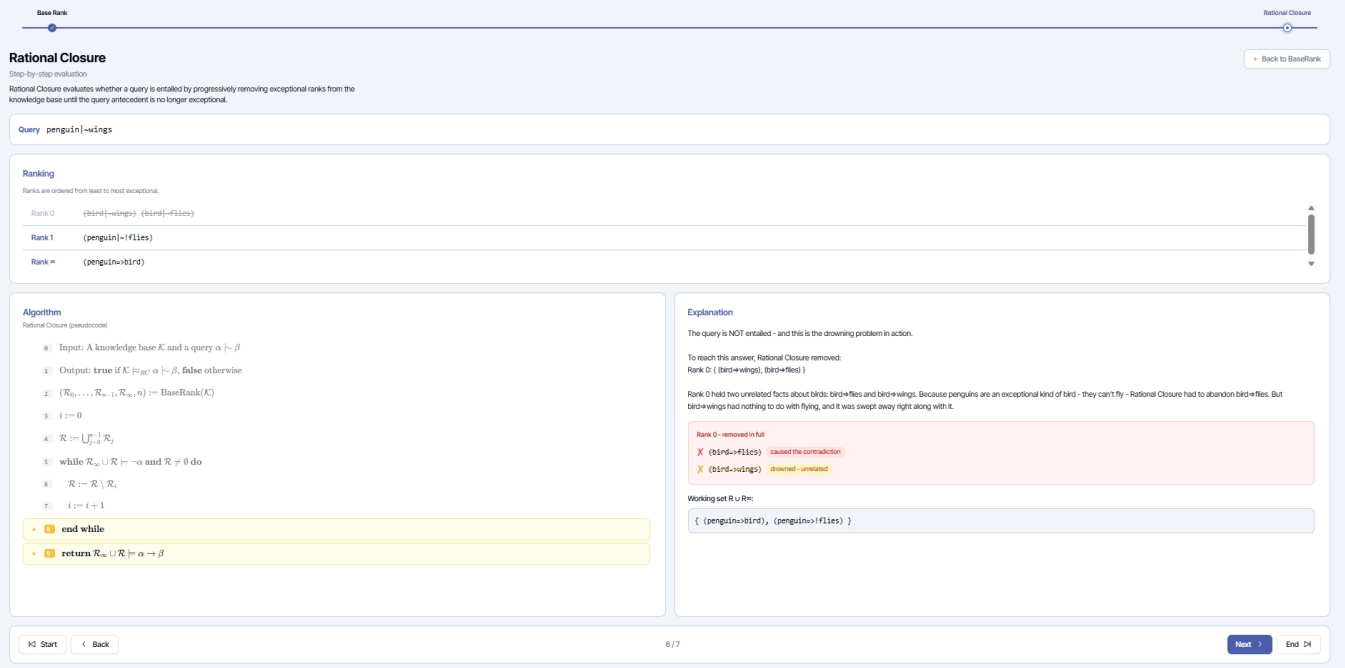
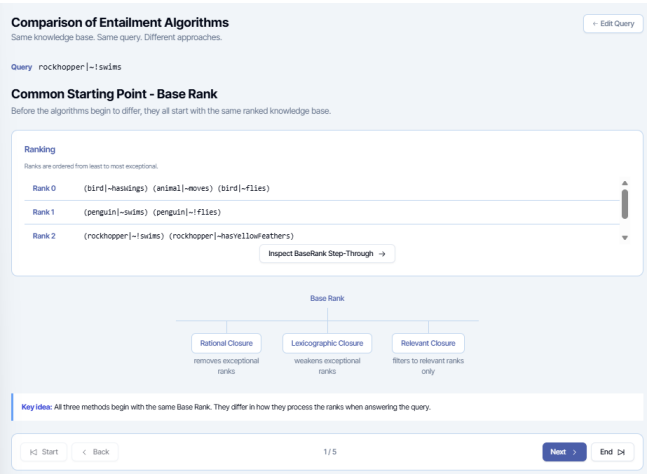
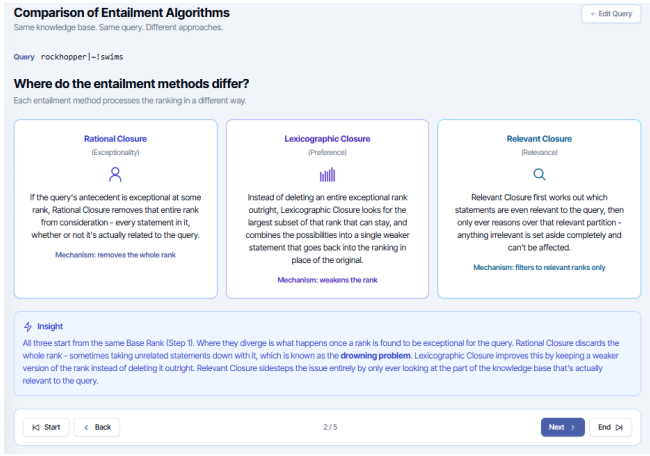


Figure 15: Predefined drowning-problem example and its dedicated explanation.

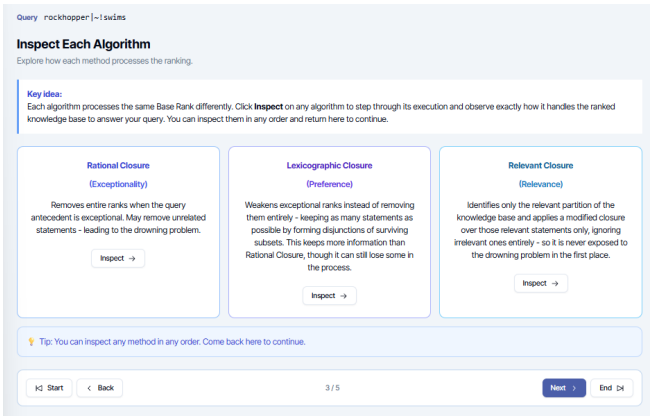
D.5 Comparison of Entailment Algorithms



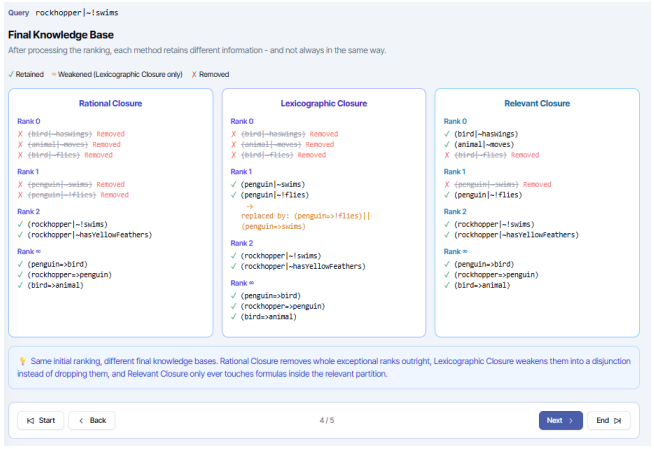
(a) Step 1: shared BaseRank.



(b) Step 2: introduction of the differences between entailment methods.



(a) Step 3: access to the individual algorithm walkthroughs.



(b) Step 4: visual comparison of how ranked information is treated.

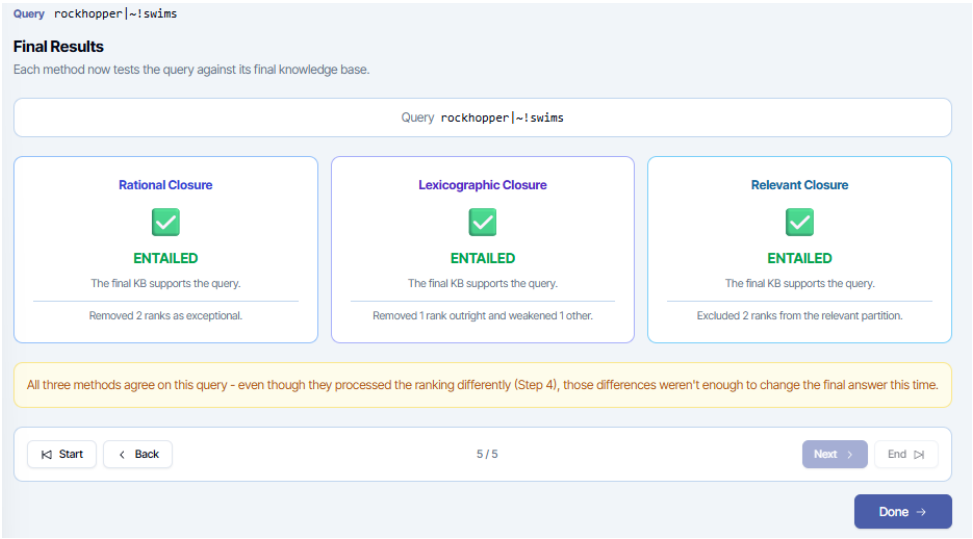


Figure 18: Step 5 of the comparison walkthrough, showing the final entailment results and whether the methods agree.